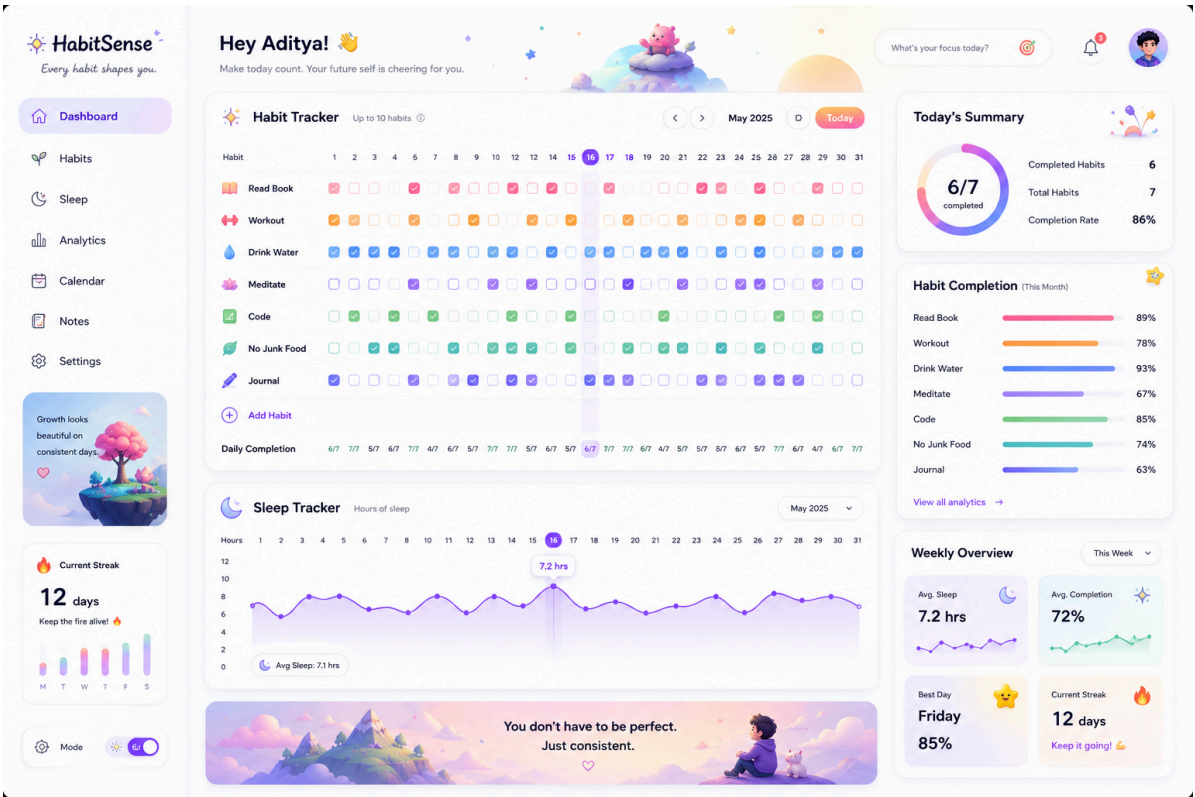


HabitSense

Every habit shapes you.

Build Methodology & Product Specification

Phase 1 — Website Foundation



Reference UI — Dashboard (target visual quality bar for this build)

Prepared for: Palak
Document version: 1.0
Last updated: June 25, 2026

Table of Contents

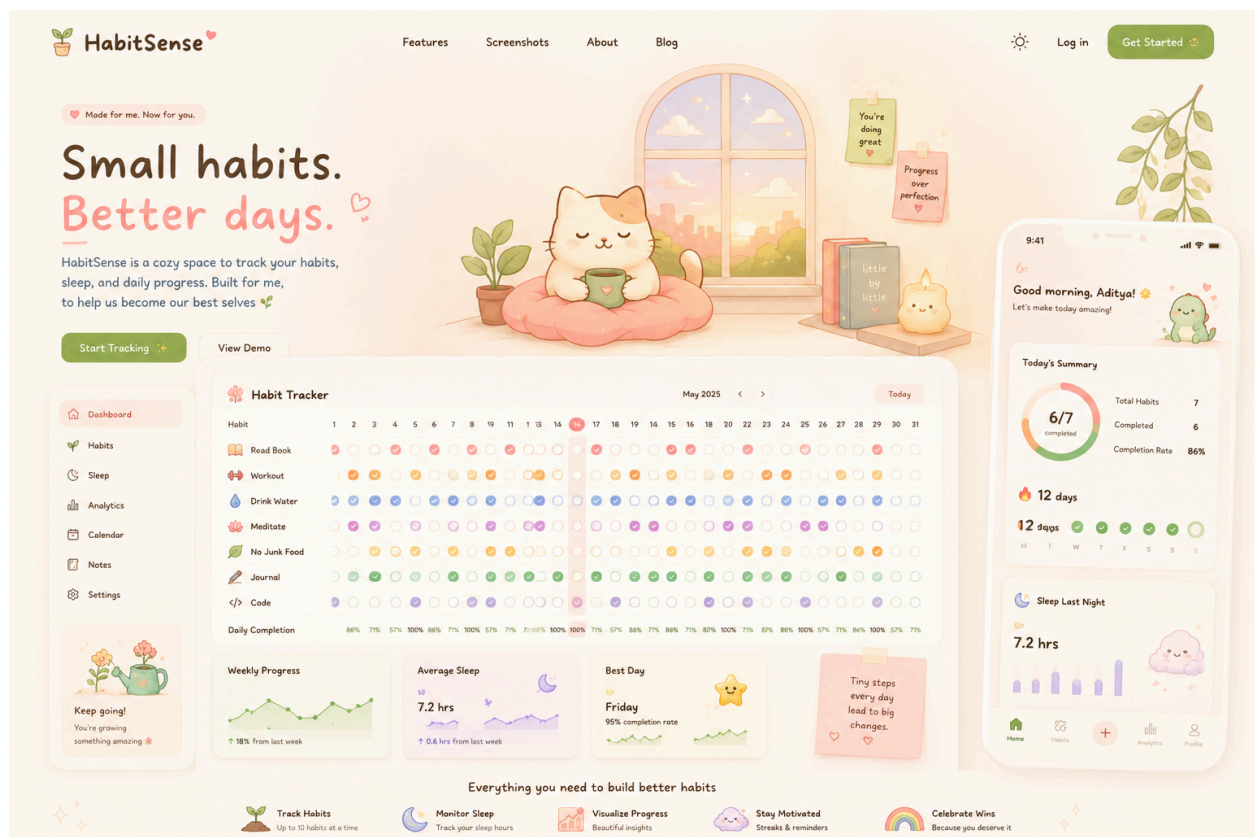
1. Product Overview

1.1 What HabitSense Is

HabitSense is a personal habit, sleep, and routine tracker built as a warm, cozy alternative to clinical productivity dashboards. The product thesis is simple: consistency tools should feel encouraging, not like another spreadsheet to maintain. This document covers Phase 1 only — a fully functional, polished web application — while keeping the architecture ready for the cloud, mood-tracking, and desktop phases that follow.

1.2 Design Language (from the reference UI)

Both reference screens below define the visual contract for this build. Every component spec in Section 4 traces back to a pattern visible in these images — color-coded habit rows, soft rounded checkboxes, pastel gradient banners, and small illustrated companions that carry the emotional tone of the app.



Reference UI — Marketing / Landing Page

Key visual signatures to preserve

- Each habit gets its own accent color (rose for Read Book, orange for Workout, blue for Drink Water, purple for Meditate, green for Code, teal for No Junk Food) — color is the primary way users distinguish rows at a glance.
- Checkboxes are circular, not square, and use a soft filled state rather than a hard checkmark-in-a-box.

- Hero / banner sections use warm gradient backgrounds (peach → lavender → sky) paired with a single small illustrated character — never busy clip-art collages.
- Card corners are generously rounded (16–24px), with soft drop shadows rather than hard borders.
- Empty states and sidebars carry a small motivational illustration + one short encouraging line, e.g. “Growth looks beautiful on consistent days.”
- Typography is friendly but not cartoonish — a clean geometric sans (the spec calls for this), with playful illustration providing the personality instead of the type.

1.3 Goals for This Document

- Lock down a buildable, unambiguous specification so implementation doesn’t stall on undecided details.
- Define a data architecture that survives the move from LocalStorage (Phase 1) to Supabase (Phase 2) without a rewrite.
- Resolve the open questions flagged in the original spec review (streak rules, mobile grid layout, PWA scope, build order).
- Give a week-by-week build plan that a single developer can execute against.

2. Scope & Guiding Principles

2.1 In Scope (Phase 1 — Website)

Area	Phase 1 Scope
Habit Tracking	Create/edit up to 10 habits, monthly grid (1–31), checkbox completion, completion %, streaks
Sleep Tracking	Daily sleep hours log, line chart trend, history + average
Analytics	Daily completion score, per-habit stats, weekly/monthly summaries
Storage	LocalStorage only, versioned schema, swappable adapter interface
Platform	Responsive website + installable PWA (offline read of existing data)

2.2 Explicitly Out of Scope (Future Roadmap)

Phase	Deferred Features
Phase 2	Supabase integration, authentication, real cloud sync
Phase 3	Mood tracking, notes/reflections, browser extension for screen time
Phase 4	Tauri desktop app — Windows, macOS, Linux

Why this boundary matters

Every Phase 2+ feature touches the data layer (auth changes who owns the data, sync changes how it's written, mood tracking adds a new entity type). Section 3 defines the storage abstraction precisely so that none of this requires touching Phase 1 UI code — only swapping the adapter implementation underneath it.

2.3 Guiding Principles

- Calm over clever. If a micro-interaction doesn't reinforce "you're doing fine," cut it.
- One adapter, many backends. The UI never talks to localStorage directly — it talks to a StorageAdapter interface (Section 3.2).
- Mobile is a different layout, not a shrunk one. The 10×31 grid does not survive a naive responsive squeeze (Section 4.3).
- Ship the dashboard last. It's a consumer of habit + sleep data, so it should be the last screen built, not the first (Section 6).

3. Data Architecture

3.1 Core Entities

Four entities cover the full Phase 1 feature set. Each is intentionally flat and serializable — no nested objects beyond one level — so they map cleanly onto both LocalStorage JSON today and Postgres rows in Phase 2.

Habit

```
type Habit = {
  id: string;           // uuid
  name: string;         // "Read Book"
  icon: string;         // lucide icon key, e.g. "book-open"
  color: HabitColor;    // see palette below – drives row + chip color
  createdAt: string;    // ISO date, first selectable day in the grid
  archivedAt: string | null;
  sortOrder: number;
};

type HabitColor =
  | "rose" | "orange" | "sky" | "violet" | "green" | "teal" | "amber" | "pink" | "indigo" |
  "lime";
```

HabitEntry

```
type HabitEntry = {
  habitId: string;
  date: string;         // "YYYY-MM-DD", local calendar day
  status: "done" | "missed" | "unmarked";
};
// Stored sparsely – only "done" and explicit "missed" entries are written.
// Absence of a record means "unmarked," not "missed." This distinction
// is what Section 3.4 uses to define streak behavior correctly.
```

SleepEntry

```
type SleepEntry = {
  date: string;         // "YYYY-MM-DD"
  hours: number;        // 0-24, 0.25 step (15 min granularity)
  loggedAt: string;     // ISO timestamp, for "last edited" purposes
};
```

AppMeta

```
type AppMeta = {
  schemaVersion: number; // start at 1 – see migration strategy 3.5
  userDisplayName: string | null;
  themeMode: "light" | "dark";
  onboardingCompletedAt: string | null;
};
```

};

3.2 The StorageAdapter Interface

This is the single most important architectural decision in Phase 1. Every read and write in the app goes through this interface. The LocalStorage implementation is a detail; the interface is the contract that makes Phase 2's Supabase migration a swap, not a rewrite.

```
interface StorageAdapter {
  // Habits
  getHabits(): Promise<Habit[]>;
  saveHabit(habit: Habit): Promise<void>;
  deleteHabit(habitId: string): Promise<void>;

  // Entries
  getEntries(range: { from: string; to: string }): Promise<HabitEntry[]>;
  setEntryStatus(habitId: string, date: string, status: HabitEntry["status"]): Promise<void>;

  // Sleep
  getSleepEntries(range: { from: string; to: string }): Promise<SleepEntry[]>;
  setSleepEntry(date: string, hours: number): Promise<void>;

  // Meta
  getMeta(): Promise<AppMeta>;
  updateMeta(patch: Partial<AppMeta>): Promise<void>;

  // Lifecycle
  exportAll(): Promise<ExportBundle>; // for backup / future migration
  importAll(bundle: ExportBundle): Promise<void>;
}
```

Phase 1 ships exactly one implementation:

- LocalStorageAdapter — implements every method above by reading/writing a single namespaced JSON blob in window.localStorage, keyed as habitsense:v1.

Phase 2 adds a second implementation without touching a single component:

- SupabaseAdapter — same interface, Postgres-backed, with the same method signatures. The app's React Query hooks (Section 4.2) call the interface, never the implementation, so the swap is a one-line change in the provider.

Implementation rule

No component, hook, or page may import localStorage directly. All access goes through useStorage(), a context hook that returns whichever StorageAdapter is currently active. This is the rule most likely to be skipped under deadline pressure — and the one that costs the most later if it is.

3.3 Storage Schema (LocalStorage JSON shape)

```
{
  "schemaVersion": 1,
  "meta": { "userDisplayName": "Palak", "themeMode": "light", "onboardingCompletedAt":
"2026-06-01T00:00:00Z" },
  "habits": [
    { "id": "h1", "name": "Read Book", "icon": "book-open", "color": "rose",
      "createdAt": "2026-06-01", "archivedAt": null, "sortOrder": 0 }
  ],
  "entries": [
    { "habitId": "h1", "date": "2026-06-03", "status": "done" }
  ],
  "sleep": [
    { "date": "2026-06-03", "hours": 7.25, "loggedAt": "2026-06-04T07:10:00Z" }
  ]
}
```

This entire object is stored under one key. Reads parse it once per session and cache in memory; writes debounce at 300ms and re-serialize the whole object — simple, and entirely sufficient at this data volume (10 habits × 365 days is ~3,650 small records a year).

3.4 Streak Rules (resolved)

This was left ambiguous in the original spec. Resolved definition for Phase 1:

1. A day counts toward a streak only if it is explicitly marked “done.” An unmarked day is neutral, not a break — it simply isn’t counted yet.
2. A streak breaks the first time a past day is explicitly marked “missed,” or when a full calendar day has elapsed with no “done” entry and the habit existed on that day.
3. Today is never treated as a break-in-progress — the streak calculation only evaluates days strictly before today, so users aren’t penalized mid-day.
4. Both Current Streak and Longest Streak are tracked per habit and computed on read (not stored), since they’re a pure function of the entries array and recomputing is cheap at this scale.
5. Backfilling past days is allowed — users can mark yesterday or last week as done, and the streak recalculates accordingly. This matches the “no pressure” tone of the product.

3.5 Schema Versioning & Migration Strategy

schemaVersion exists from day one so Phase 2 has a hook to migrate from. The pattern:

- On load, the adapter checks schemaVersion against the current code’s expected version.
- If older, it runs an ordered list of migration functions (migrations[1→2], migrations[2→3]...) before the app reads any data.
- Phase 2’s Supabase migration becomes: read via LocalStorageAdapter.exportAll(), run schema migration if needed, import via SupabaseAdapter.importAll(). No manual data wrangling.

4. UI & Component Specification

This section translates the reference screenshots into a buildable component breakdown. Every component below is visible in one of the two reference images.

4.1 Color System

Extracted from the reference UI. These are the per-habit accent colors plus the core neutral and gradient palette used across cards and banners.

Token	Hex / Use
habit.rose	#F4778E — e.g. “Read Book” row
habit.orange	#F2994A — e.g. “Workout” row
habit.sky	#4FA8E0 — e.g. “Drink Water” row
habit.violet	#9B7FD4 — e.g. “Meditate” row
habit.green	#5FB87B — e.g. “Code” row
habit.teal	#3FB8A6 — e.g. “No Junk Food” row
surface.bg	#FFFFFF cards on #FBF8FD app background
surface.muted	#F8F5FB — zebra striping, subtle panels
text.heading	#352B42
text.body	#2B2433
text.muted	#6B6275
gradient.hero	linear-gradient(135deg, #FCE0D6 0%, #E6D9F5 55%, #D6E8F5 100%)

4.2 Application Shell

Matches the dashboard reference: a fixed left sidebar (Dashboard, Habits, Sleep, Analytics, Calendar, Notes, Settings), a top greeting bar with a contextual “Hey [name]” message, and a notification + avatar cluster top-right.

Sidebar

- Logo + tagline at top (“HabitSense — Every habit shapes you.”).
- Nav items with icon + label, active state shown as a soft lavender pill background, matching the “Dashboard” highlighted state in the reference.

- A motivational illustration card pinned near the bottom (tree/growth illustration in the reference) — this is a static encouragement card, not functional, and can rotate between 4–6 pre-written illustrated messages.
- A “Current Streak” mini-card below that, showing the longest active streak across all habits with a small per-day bar visualization (M–S).
- Light/Dark mode toggle pinned at the very bottom.

Top bar

- Greeting text is time-of-day aware: “Good morning,” “Good afternoon,” “Good evening,” followed by the user’s name and a waving emoji.
- Subtext line rotates between a small set of encouraging copy lines (“Make today count. Your future self is cheering for you.” is the reference example).
- A “What’s your focus today?” input — Phase 1 treats this as a lightweight daily intention note, stored as a single string per date in AppMeta-adjacent storage (not a full Notes feature, which is Phase 3).

4.3 Habit Tracker Grid

The core component. Reference shows a table: habit name + icon in the left column, 31 day-columns, one circular checkbox per cell, color-coded per habit, with a “Daily Completion” footer row showing fraction completed per day.

Desktop / tablet layout (≥ 768px)

- Exact grid as shown in the reference: sticky first column (habit name), horizontally scrollable day columns if the viewport is narrower than 31 columns.
- Month navigation via chevrons either side of the “May 2025” label, plus a “Today” button that jumps the scroll position and highlights the current date column (shown as a subtle vertical lavender band in the reference).
- Clicking a circle cycles unmarked → done → missed → unmarked (or simply unmarked ↔ done if a simpler v1 is preferred — see Section 7 build order).
- “Add Habit” row stays pinned beneath the last habit, matching the reference’s dashed-style add affordance.

Mobile layout (< 768px) — resolved gap from original spec

A 10×31 grid does not fit a phone screen without becoming the “spreadsheet-like” interface the spec explicitly warns against. Phase 1 ships a distinct mobile view, not a shrunk grid:

- Mobile shows one day at a time: a vertical list of that day’s habits, each as a row with name, icon, and a single large circular checkbox.
- A horizontal date strip (7 visible days, swipeable) sits above the list for quick day-switching — this is the same interaction pattern as the mobile mock’s “Today’s Summary” screen on the right side of the landing page reference.

- A small calendar icon opens a month-grid date picker for jumping to a specific day, rather than trying to render all 31 days inline.
- Both layouts read/write the same HabitEntry records — this is a view-layer split only, not a data-layer one.

Completion percentage & streak display

- Per-habit completion % (right panel in the dashboard reference) = $\text{done days} \div \text{days elapsed since habit creation within the selected month}$.
- Daily Completion footer = $\text{done habits that day} \div \text{total active (non-archived) habits on that day}$.
- Streak flame icon + day count shown in the sidebar card, computed per Section 3.4 rules.

4.4 Sleep Tracker

Reference shows a line chart (Recharts) with day-of-month on the x-axis and hours on the y-axis, a hover tooltip showing exact hours for the selected day, and an “Avg. Sleep” pill anchored bottom-left of the chart.

- Logging interaction: tapping/clicking a date on the chart x-axis (or a small “+ Log sleep” affordance) opens an inline numeric stepper for hours, in 15-minute increments, defaulting to last night’s typical value.
- Chart line uses the lavender accent (#8B6FC2-family) gradient-filled beneath, matching the reference’s soft purple area fill.
- Month selector dropdown, top-right of the card, matches the reference’s “May 2025” dropdown.
- History view (separate from the chart) is a simple reverse-chronological list, paginated by month, available from the Sleep nav item.

4.5 Today's Summary & Weekly Overview (right rail)

- Donut chart (Recharts PieChart) showing completed/total habits today, with the fraction as center text (‘6/7 completed’ in the reference) — gradient ring from rose to lavender.
- Habit Completion list: horizontal progress bars per habit for the current month, color matching that habit’s accent, sorted by the habit’s sortOrder (not by completion rate, to keep positions stable for the user).
- Weekly Overview: four small stat cards — Avg. Sleep, Avg. Completion, Best Day, Current Streak — each with a tiny sparkline or icon, exactly as laid out in the reference.

4.6 Analytics Dashboard

Built last (Section 6), since it aggregates data the other two features produce. Composition:

- Weekly Progress line chart (small multiple of the main sleep chart pattern, applied to overall completion % instead of sleep hours).
- Per-habit statistics table: completion %, current streak, longest streak, total days logged — sortable by column.

- Monthly summary card: best day, most consistent habit, most-missed habit, total active days.
- All charts reuse a single shared <TrendChart> component (Section 5.3) parameterized by data series, to avoid four bespoke chart implementations.

4.7 Empty, Loading, and Onboarding States — resolved gap from original spec

First-run experience was missing from the original spec. Defined now:

- First load with zero habits: the grid area is replaced with a centered illustration (in the visual language of the sidebar’s growth-tree card) plus a single “Create your first habit” button. No empty table is ever shown.
- First load with habits but zero sleep entries: the Sleep Tracker chart area shows a friendly prompt (“Log your first night to start your trend”) instead of a flat empty chart.
- Loading states use skeleton shimmer blocks shaped like the real components (grid rows, chart area) rather than spinners, to avoid layout jump.
- A lightweight 3-step onboarding modal on first visit: name entry → pick 3 starter habits from a suggested list (or skip) → done. This populates Habit records and sets `onboardingCompletedAt`.

5. Application Architecture

5.1 Folder Structure

Feature-based organization, resolved from the original spec's unstated "scalable architecture" goal. Shared UI primitives stay separate from feature logic so Phase 2—4 additions are additive, not invasive.

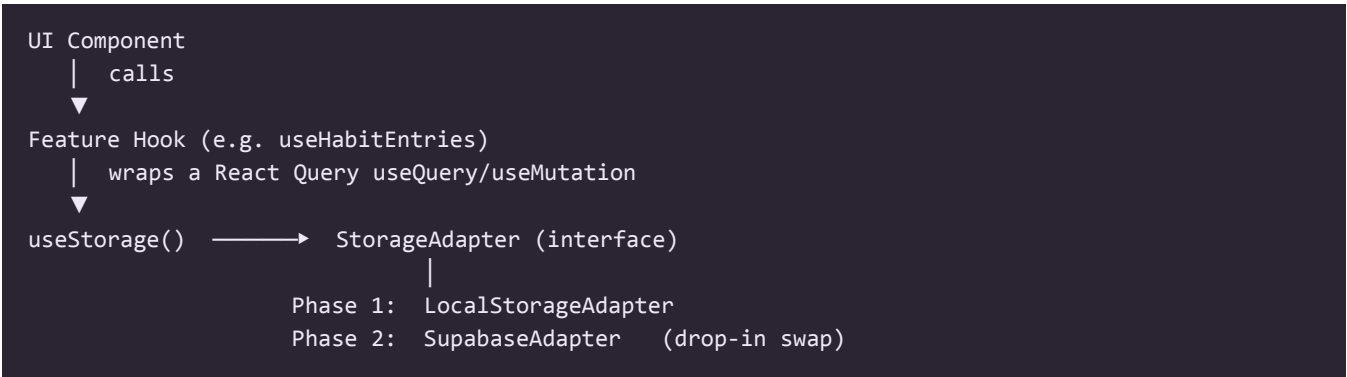
```

habitsense/
├── app/                                # Next.js App Router
│   ├── (dashboard)/
│   │   ├── page.tsx                  # Dashboard (home)
│   │   ├── habits/page.tsx
│   │   ├── sleep/page.tsx
│   │   ├── analytics/page.tsx
│   │   └── layout.tsx                # Sidebar + topbar shell
│   ├── layout.tsx                    # Root layout, theme provider
│   └── globals.css
├── features/
│   ├── habits/
│   │   ├── components/              # HabitGrid, HabitRow, AddHabitDialog...
│   │   ├── hooks/                   # useHabits, useStreak, useHabitEntries
│   │   └── utils/                    # streak.ts, completion.ts
│   ├── sleep/
│   │   ├── components/              # SleepChart, SleepLogger, SleepHistory
│   │   └── hooks/                   # useSleepEntries
│   └── analytics/
│       ├── components/              # TrendChart, StatCard, SummaryPanel
│       └── hooks/                   # useAnalytics
├── components/
│   ├── ui/                          # shadcn/ui primitives (button, dialog, etc.)
│   └── shared/                       # Sidebar, TopBar, EmptyState, Skeletons
├── lib/
│   ├── storage/
│   │   ├── types.ts                 # Habit, HabitEntry, SleepEntry, AppMeta
│   │   ├── storage-adapter.ts        # interface definition
│   │   ├── local-storage-adapter.ts
│   │   ├── migrations.ts
│   │   └── storage-provider.tsx      # React context exposing useStorage()
│   └── utils/                        # date helpers, formatters
├── public/
│   └── illustrations/                # SVGs in the reference's art style
├── next.config.js
├── tailwind.config.ts
└── manifest.json                     # PWA manifest

```

5.2 Data Flow

Single direction, no surprises: components call feature hooks → hooks call useStorage() → adapter reads/writes → React Query cache invalidates → components re-render.



React Query (TanStack Query) is added to the stack here — it isn’t in the original tech list, but it’s the difference between “clean architecture” as a stated goal and clean architecture as a reality. It gives caching, optimistic updates (checkbox taps feel instant), and a consistent loading/error pattern for free, and its query-key model maps directly onto swapping adapters later.

5.3 Key Shared Components

Component	Responsibility
<StorageProvider>	Wraps the app; instantiates the active adapter; exposes useStorage()
<HabitGrid>	Desktop/tablet monthly grid — renders <HabitRow> × N
<HabitDayList>	Mobile single-day view — the resolved mobile pattern from 4.3
<TrendChart>	Generic Recharts line/area chart, reused for sleep + analytics trends
<StatCard>	Small metric card (icon, value, label, optional trend) — used across summary rails
<EmptyState>	Illustration + message + CTA, used for zero-habit / zero-sleep-data states
<StreakBadge>	Flame icon + day count, computed via the Section 3.4 streak utility

6. Build Methodology

Resolves the original spec's missing build-order guidance. The sequencing logic: storage layer first (everything depends on it), habits before sleep (it's the primary feature), analytics last (it consumes both).

6.1 Recommended Build Order

Stage 0 — Foundation (no visible UI yet)

6. Scaffold Next.js + TypeScript + Tailwind + shadcn/ui. Configure the color tokens from Section 4.1 in `tailwind.config.ts`.
7. Write `lib/storage/types.ts` and `storage-adapter.ts` — the interface, fully typed, before any implementation exists.
8. Implement `LocalStorageAdapter` against that interface, plus the `StorageProvider` context.
9. Write unit tests for the adapter (get/set/delete round-trips) before building UI against it — this catches schema mistakes while they're still cheap to fix.

Stage 1 — Habits (the core loop)

10. Build `<HabitGrid>` and `<HabitRow>` against mock data first, matching the reference pixel-for-pixel before wiring real storage.
11. Wire `useHabits` / `useHabitEntries` hooks to the real adapter.
12. Implement the checkbox toggle interaction with optimistic updates.
13. Add streak calculation (Section 3.4) and the `<StreakBadge>`.
14. Build the mobile `<HabitDayList>` view and the responsive breakpoint switch.
15. Add the Add/Edit/Archive habit dialog (10-habit cap enforced here).

Stage 2 — Sleep

16. Build the sleep logging interaction (numeric stepper) against mock data.
17. Wire `<TrendChart>` to real sleep entries; add the average-sleep pill.
18. Build the sleep history list view.

Stage 3 — Dashboard composition

19. Now that habits + sleep produce real data, assemble Today's Summary, Habit Completion bars, and Weekly Overview — these are read-only views over Stage 1–2 data, not new data sources.

Stage 4 — Analytics

20. Build the analytics aggregation utilities (weekly/monthly rollups) as pure functions, unit-tested independently of any chart.
21. Wire the Analytics page using the same `<TrendChart>` and `<StatCard>` components already built.

Stage 5 — Polish pass

- 22. Empty states, onboarding flow, skeleton loaders (Section 4.7).
- 23. Framer Motion micro-interactions: checkbox fill animation, streak flame pulse, confetti-lite on hitting 100% for a day.
- 24. PWA manifest + service worker for offline read access (Section 6.3).
- 25. Accessibility pass: keyboard navigation through the grid, ARIA labels on checkboxes, color-blind-safe redundant icons alongside the color coding.

6.2 Suggested Timeline

Week	Focus
Week 1	Stage 0 — foundation, storage adapter, design tokens
Week 2	Stage 1 — habit grid (desktop), checkbox interactions, streaks
Week 3	Stage 1 cont. — mobile day view, habit CRUD, Stage 2 sleep tracker
Week 4	Stage 3 — dashboard composition, Stage 4 analytics
Week 5	Stage 5 — polish, onboarding, PWA, accessibility, deploy to Vercel

6.3 PWA Scope (resolved gap from original spec)

“PWA” was listed under Platform with no concrete requirements. Defined minimally for Phase 1:

- manifest.json with app name, icons (192/512px, matching the reference’s sparkle mark), theme color, and standalone display mode — enables “Add to Home Screen.”
- A service worker caching the app shell (JS/CSS/fonts) so the app opens instantly and previously-loaded data is viewable offline.
- Writes (checking off a habit, logging sleep) work offline since they only touch LocalStorage — no network dependency exists in Phase 1, so there is no offline-sync queue to build yet. That complexity is deferred to Phase 2, when Supabase introduces an actual network round-trip.

7. Tech Stack Reference

Layer	Choice
Framework	Next.js (App Router) + TypeScript
Styling	Tailwind CSS + shadcn/ui component primitives
Animation	Framer Motion
Charts	Recharts (line/area for trends, pie/radial for the completion donut)
Data fetching / cache	TanStack Query — added in this doc; sits between hooks and the adapter
Storage (Phase 1)	Browser LocalStorage, behind StorageAdapter
Storage (Phase 2)	Supabase — same adapter interface, new implementation
Deployment	Vercel
Platform	Responsive website + installable PWA

7.1 Notable additions vs. the original tech list

- TanStack Query — not originally listed, added because it's what makes the StorageAdapter swap in Phase 2 actually painless rather than theoretically painless.
- lucide-react for icons — pairs naturally with shadcn/ui and matches the simple line-icon style visible in the reference sidebar (home, sprout, moon, bar-chart icons).
- date-fns for all date math (streaks, month grids, week boundaries) — avoids hand-rolled calendar bugs, which are the single most common source of off-by-one errors in habit trackers.

8. Summary — Gaps Closed From the Original Spec

For traceability, every ambiguity flagged in the original spec review is resolved at a specific section in this document:

Original Gap	Resolved In
LocalStorage → future cloud sync abstraction	Section 3.2 — StorageAdapter interface
Schema versioning / migration story	Section 3.5
10×31 grid breaking on mobile	Section 4.3 — dedicated mobile day-list view
Streak rule edge cases	Section 3.4
Missing onboarding / empty states	Section 4.7
Build sequencing (analytics depends on other data)	Section 6.1, Stage 3–4
PWA requirements unspecified	Section 6.3
No stated component/folder architecture	Section 5.1
Completion % vs. daily completion score overlap	Section 4.3 — explicit formulas for each

Next step

With this document as the spec, the natural next artifact is a starter repo: Next.js scaffolded with the folder structure from Section 5.1, the StorageAdapter interface and LocalStorageAdapter implemented and tested, and the design tokens from Section 4.1 wired into Tailwind — ready for Stage 1 component work to begin directly.